

NATIONAL ECONOMICS UNIVERSITY – FACULTY OF DATA SCIENCE
AND AI

DSAI1002: Data Structures and Algorithms with Python

LECTURE NOTES: PART 03

Algorithms and Complexity Analysis

Author / Instructor: Dr. Minh D. Vu | Term: Fall 2026

Comprehensive Guide to Algorithm Design, Asymptotic Notations, Loop Analysis, Recurrences, and Amortized Cost

Executive Roadmap of Part 03

This monograph provides an in-depth, self-contained reference for **Part 03: Algorithms and Complexity Analysis**. The curriculum is partitioned into five fundamental pillars:

- Algorithmic Foundations & Design Process:** Formal definitions, the algorithm–program distinction, core characteristics, flowcharting, pseudocode, the 8-stage engineering workflow, and a priori versus a posteriori evaluation.
- Asymptotic Notations ($O, \Omega, \Theta, o, \omega$):** Mathematical definitions, limits, dominance hierarchies, simplification rules (sum, product), and space complexity versus auxiliary space.
- Non-Recursive Loop Complexity:** Systematic techniques for linear, logarithmic, triangular, geometric, and harmonic loop structures, including best-, worst-, and average-case analysis.
- Divide-and-Conquer Recurrences & Master Theorem:** Formulating recurrence relations, recursion tree modeling, standard Master Theorem cases, the Extended Master Theorem ($\Theta(n^k \log^p n)$), subtract-and-conquer recurrences, variable transformations, and induction proofs.
- Amortized Analysis:** Aggregate, accounting (banker’s), and potential (physicist’s) methods applied to dynamic array resizing, multipop stacks, and binary counters.

Contents

1	What is an Algorithm? Foundations of Algorithmic Thinking	2
1.1	Intuitive Notion and Formal Definition	2
1.2	Algorithms versus Programs	2
1.3	Essential Characteristics of a Valid Algorithm	2
1.4	Representation of Algorithms	3
1.5	The 8-Stage Engineering Workflow for Algorithm Design	4
1.6	A Priori versus A Posteriori Analysis	4
1.7	Why Wall-Clock Time Fails as a Theoretical Metric	5
2	Asymptotic Notations and Formal Complexity Foundations	6
2.1	The Philosophy of Asymptotics: Focus on Growth Rates	6
2.2	The Three Fundamental Asymptotic Notations	6
2.3	The Limit Criterion for Asymptotic Dominance	7
2.4	Standard Hierarchy of Complexity Classes	7

2.5	Fundamental Operational Rules for Asymptotic Arithmetic	8
2.6	Space Complexity versus Auxiliary Space	8
3	In-Depth Code & Loop Complexity Analysis	10
3.1	Linear Increment Loops: $\Theta(n)$	10
3.2	Logarithmic Loops (Multiplication / Division): $\Theta(\log n)$	10
3.3	Triangular Nested Loops: $\Theta(n^2)$	10
3.4	Geometric Series Loop Progressions: $\Theta(n)$	10
3.5	Harmonic Number Loops: $\Theta(n \log n)$	11
3.6	Quadratic Loop Conditions: $\Theta(\sqrt{n})$	11
3.7	Best-Case, Worst-Case, and Average-Case Behavior	11
4	Divide-and-Conquer Recurrences and the Master Theorem	12
4.1	Formulating Recurrence Relations	12
4.2	The Recursion Tree Method	12
4.3	The Standard Master Theorem (CLRS)	13
4.4	The Extended Master Theorem (Polylogarithmic Form)	13
4.5	When the Master Theorem Cannot Be Applied (7 Classic Traps)	14
4.6	Subtract-and-Conquer Recurrences	14
4.7	Advanced Recurrences via Variable Substitution	15
5	Amortized Analysis Foundations & Case Studies	16
5.1	The Motivation for Amortized Analysis	16
5.2	The Three Standard Amortized Analysis Techniques	16
5.2.1	1. The Aggregate Method	16
5.2.2	2. The Accounting Method (Banker's Method)	16
5.2.3	3. The Potential Method (Physicist's Method)	16
5.3	Case Study 1: Dynamic Array (Python 'list') Resizing	17
5.3.1	Analysis via Aggregate Method	17
5.3.2	Analysis via Potential Method	17
5.4	Case Study 2: Stack with MultiPop	18
6	Worked Problems and Solutions	19
6.1	Problem 1: Growth Rate Ranking	19
6.2	Problem 2: Non-Trivial Loop Complexity	19
6.3	Problem 3: Applying the Master Theorem	19
7	Quick Reference Summary Sheet	21
7.1	Essential Summation Formulas	21
7.2	Master Theorem Decision Matrix	21
7.3	Amortized Cost Comparison Matrix	21

1. What is an Algorithm? Foundations of Algorithmic Thinking

1.1 Intuitive Notion and Formal Definition

In everyday life, problem solving involves an ordered sequence of actionable steps: preparing a recipe, finding a contact in a phone directory, or navigating through urban traffic. When these instructions are stated with unambiguous precision, finite length, and guaranteed executability, they embody **algorithmic thinking**.

Definition 1.1 (Algorithm)

An **algorithm** is a finite sequence of well-defined, computer-implementable instructions designed to solve a specific class of computational problems, perform an automated task, or compute a mathematical function. Formally, an algorithm accepts a set of inputs, performs a deterministic or randomized sequence of state transitions, and produces a well-defined output after a finite number of computational steps.

The computational pipeline can be abstracted into the canonical three-stage model:



1.2 Algorithms versus Programs

A common beginner mistake is conflating an *algorithm* with a *computer program*. An algorithm represents the abstract mathematical idea and logical workflow, whereas a program is a concrete realization written in a specific language (e.g., Python, C++, Java) executable by hardware.

Dimension	Algorithm	Program
Nature	Conceptual idea, mathematical process	Concrete linguistic implementation
Language	Universal, syntax-independent (pseudocode/math)	Python, C++, Rust, Java, etc.
Level of Detail	High-level logic, invariants, convergence	Low-level syntax, types, memory, libraries
Target Audience	Human mathematicians, software engineers	Translators (compilers, interpreters), CPUs
Lifespan	Decades or centuries (e.g., Euclid’s GCD: 300 BC)	Subject to library deprecation and syntax changes

1.3 Essential Characteristics of a Valid Algorithm

According to Donald Knuth (*The Art of Computer Programming*), every valid computational algorithm must satisfy six core criteria:

- Definiteness (Unambiguity):** Each instruction must be clear, precise, and have exactly one possible interpretation. Ambiguous directives such as “pick a large number” or “sort the items quickly” are strictly invalid.
- Defined Input:** An algorithm has zero or more formally specified inputs, each with well-defined domains, data types, and structural preconditions.
- Defined Output:** An algorithm must produce at least one output or measurable effect that has a specified relationship to the input.

4. **Finiteness:** The algorithm must terminate after a finite number of discrete operations for any valid input. Infinite loops violating termination conditions do not constitute algorithms.
5. **Effectiveness (Feasibility):** Every operation must be primitive enough that it could, in principle, be executed exactly by a person using paper and pencil in a finite amount of time (e.g., basic integer arithmetic, pointer dereference).
6. **Correctness:** For all valid input instances, the algorithm produces the correct output as defined by the specification.

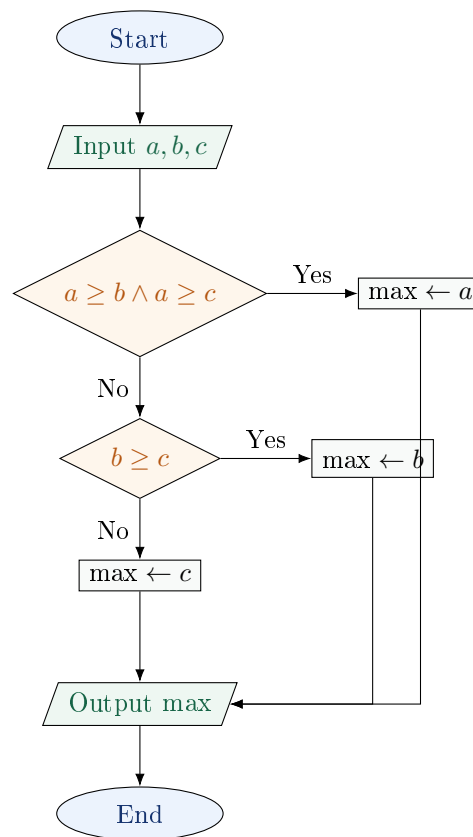
Determinism versus Randomization

Classical algorithms are **deterministic**: given an identical input, the algorithm follows the identical execution trajectory and produces identical output. However, modern computer science embraces **randomized algorithms** (e.g., Randomized QuickSort, Monte Carlo and Las Vegas algorithms) which make random choices during execution. Randomized algorithms remain genuine algorithms provided their probability distribution, termination guarantees, and correctness metrics are rigorously characterized.

1.4 Representation of Algorithms

Algorithms are communicated across three primary levels of abstraction:

1. **Natural Language:** Intuitive and accessible, but inherently prone to linguistic ambiguity, verbosity, and lack of formal structural control.
2. **Flowcharts:** Visual diagrams utilizing standardized ANSI symbols (Ovals for Start/End, Parallelograms for Input/Output, Rectangles for Processing, and Diamonds for Decision branching). Excellent for visual branching logic, but unmanageable for large systems.
3. **Pseudocode:** A structured, high-level informal notation that resembles programming language constructs while omitting syntax overhead (such as semicolons, memory management, and library imports). Pseudocode is the gold standard in academic literature.



1.5 The 8-Stage Engineering Workflow for Algorithm Design

Developing robust computational solutions requires a systematic discipline:

1. **Stage 1: Problem Definition.** Clearly identify what mathematical mapping is needed.
2. **Stage 2: Input Specification.** Define parameter domains, scale boundaries ($n \leq 10^5$), and data formats.
3. **Stage 3: Output Specification.** Define exact return values or side effects.
4. **Stage 4: Constraint Identification.** Note real-world constraints (e.g., $O(n \log n)$ time budget, $O(1)$ extra space).
5. **Stage 5: Conceptual Formulation.** Devise algorithmic paradigms (Divide-and-Conquer, Greedy, Dynamic Programming).
6. **Stage 6: Formal Pseudocode & Invariants.** Detail loop logic, base cases, and invariants.
7. **Stage 7: Edge-Case Verification.** Rigorously test degenerate cases (empty arrays, duplicates, singletons, extreme values).
8. **Stage 8: Theoretical Complexity Analysis.** Establish a priori bounds for time and auxiliary space.

1.6 A Priori versus A Posteriori Analysis

Evaluating performance is performed through two complementary methodologies:

Criterion	A Priori Analysis (Theoretical)	A Posteriori Analysis (Empirical)
Phase	Before software implementation	After implementation on physical hardware
Method	Mathematical counting & asymptotic bounds	Profiling, CPU timers ('timeit'), memory profilers
Hardware Dependence	Zero (Independent of CPU, OS, compiler)	High (Dependent on CPU cache, RAM speed, OS)
Goal	Categorize asymptotic growth rate as $n \rightarrow \infty$	Measure exact wall-clock runtime (seconds, MB)
Representation	Big- O , Big- Ω , Big- Θ expressions	Plots of milliseconds versus sample size N

1.7 Why Wall-Clock Time Fails as a Theoretical Metric

Beginners often ask: “*Why not just measure runtime in seconds using a timer?*” Wall-clock timing is fundamentally flawed for theoretical classification because:

- Runtimes fluctuate wildly across hardware generations (a modern laptop vs. an embedded microcontroller).
- Compiler optimizations (vectorization, loop unrolling) alter execution constants by an order of magnitude.
- Operating system scheduling, background processes, and cache misses introduce non-deterministic noise.
- A program running in 0.001 seconds on $n = 10$ may take 10^{15} years when $n = 100$ if its complexity is $O(2^n)$.

Therefore, theoretical analysis counts **basic machine operations** as a function of the input size n .

2. Asymptotic Notations and Formal Complexity Foundations

2.1 The Philosophy of Asymptotics: Focus on Growth Rates

Asymptotic analysis investigates the behavior of an algorithm's resource consumption (running time $T(n)$ or auxiliary memory $S(n)$) as the input size n approaches infinity ($n \rightarrow \infty$). Under this regime, two fundamental simplifications hold:

1. **Multiplicative Constants are Suppressed:** Whether an algorithm takes $2n$ or $50n$ operations, both scale linearly.
2. **Lower-Order Terms are Neglected:** In $f(n) = 3n^2 + 1000n + 50000$, when $n = 10^6$, the quadratic term is 3×10^{12} while $1000n = 10^9$ (less than 0.04%). The highest-order term dictates scalability.

2.2 The Three Fundamental Asymptotic Notations

Definition 2.1 (Big O – Asymptotic Upper Bound)

Let $f(n)$ and $g(n)$ be functions mapping $\mathbb{N} \rightarrow \mathbb{R}^+$. We write:

$$f(n) = O(g(n)) \quad \text{or} \quad f(n) \in O(g(n))$$

if and only if there exist strictly positive constants $c > 0$ and $n_0 \geq 1$ such that:

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0.$$

Intuition: $g(n)$ represents an asymptotic ceiling. The algorithm will perform *at most* proportional to $g(n)$ steps for sufficiently large inputs. Big- O characterizes **worst-case guarantees**.

Definition 2.2 (Big Ω – Asymptotic Lower Bound)

We write $f(n) = \Omega(g(n))$ if and only if there exist positive constants $c > 0$ and $n_0 \geq 1$ such that:

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{for all } n \geq n_0.$$

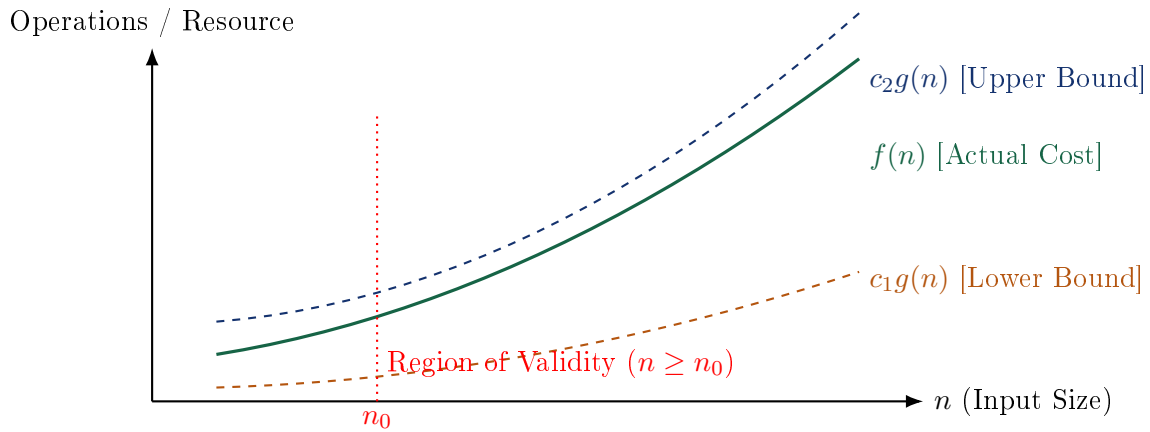
Intuition: $g(n)$ provides an asymptotic floor. The algorithm requires *at least* proportional to $g(n)$ steps. Big- Ω characterizes problem lower bounds (e.g., comparison sort requires $\Omega(n \log n)$).

Definition 2.3 (Big Θ – Asymptotically Tight Bound)

We write $f(n) = \Theta(g(n))$ if and only if there exist positive constants $c_1, c_2 > 0$ and $n_0 \geq 1$ such that:

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0.$$

Theorem: $f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)).$



2.3 The Limit Criterion for Asymptotic Dominance

Evaluating constants c and n_0 from first principles can be tedious. The **limit test** using L'Hôpital's rule offers an elegant analytical alternative:

$$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

Value of Limit L	Asymptotic Relationship	Interpretation
$L = 0$	$f(n) = o(g(n))$ and $f(n) = O(g(n))$, but $f(n) \neq \Theta(g(n))$	f grows strictly slower than g
$0 < L < \infty$	$f(n) = \Theta(g(n))$ (hence also $O(g(n))$ and $\Omega(g(n))$)	f and g have identical growth rates
$L = \infty$	$f(n) = \omega(g(n))$ and $f(n) = \Omega(g(n))$, but $f(n) \neq \Theta(g(n))$	f grows strictly faster than g

2.4 Standard Hierarchy of Complexity Classes

In asymptotic order of growth, common complexity classes are strictly ordered as follows:

$$O(1) \prec O(\log \log n) \prec O(\log n) \prec O(\sqrt{n}) \prec O(n) \prec O(n \log n) \prec O(n^2) \prec O(n^3) \prec O(2^n) \prec O(n!) \prec O(n^n)$$

Class	Formal Name	Canonical Algorithmic Example
$O(1)$	Constant	Array element index access (<code>arr[i]</code>), hash table lookup (average)
$O(\log \log n)$	Double Logarithmic	Interpolation search on uniformly distributed data, Van Emde Boas trees
$O(\log n)$	Logarithmic	Binary search in sorted array, balanced BST lookup (AVL, Red-Black)
$O(\sqrt{n})$	Sub-linear	Square root decomposition (Mo's algorithm), trial division primality
$O(n)$	Linear	Linear search, single array traversal, finding min/max
$O(n \log n)$	Linearithmic	Optimal comparison sort (MergeSort, HeapSort, QuickSort avg)
$O(n^2)$	Quadratic	Nested comparison sorts (Bubble, Selection, Insertion), all-pairs checks
$O(n^3)$	Cubic	Naive matrix multiplication ($n \times n$), Floyd-Warshall all-pairs shortest path
$O(2^n)$	Exponential	Generating all subsets (power set), recursive Fibonacci without memoization
$O(n!)$	Factorial	Generating all permutations of n elements, naive Traveling Salesperson

2.5 Fundamental Operational Rules for Asymptotic Arithmetic

1. **Sum Rule (Sequential Blocks):** If code block 1 takes $O(f(n))$ and subsequent block 2 takes $O(g(n))$:

$$T(n) = O(f(n) + g(n)) = O(\max(f(n), g(n)))$$

2. **Product Rule (Nested Loops):** If an outer loop executes $f(n)$ times and for each step the inner loop performs $g(n)$ operations:

$$T(n) = O(f(n) \cdot g(n))$$

3. **Scalar Multiplication Invariance:** For any positive constant $k > 0$, $O(k \cdot f(n)) = O(f(n))$.
4. **Transitivity:** If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

2.6 Space Complexity versus Auxiliary Space

A precise distinction must be maintained when discussing memory utilization:

- **Total Space Complexity:** The complete memory allocated by the program, which equals Input Space + Auxiliary Space.
- **Auxiliary Space:** The *extra or temporary* storage created by the algorithm during its execution to solve the problem (excluding the memory required to hold the inputs).

Auxiliary Space in Recursion: The Call Stack

A recursive implementation of factorial allocates no auxiliary arrays:

```
1 def factorial(n):  
2     if n <= 1:  
3         return 1  
4     return n * factorial(n - 1)
```

Although no user variables are allocated, each recursive call places an **activation record** (call frame containing return address, local parameters, and saved registers) onto the runtime call stack. Since the call stack reaches depth n , the **auxiliary space** is $\Theta(n)$.

3. In-Depth Code & Loop Complexity Analysis

3.1 Linear Increment Loops: $\Theta(n)$

When the loop control variable is incremented or decremented by a non-zero constant c :

```

1 i = 0
2 while i < n:
3     # O(1) body operations
4     i += c

```

The number of iterations is $k = \lceil n/c \rceil$. By dropping constant factors, $T(n) = \Theta(n)$.

3.2 Logarithmic Loops (Multiplication / Division): $\Theta(\log n)$

When the loop variable is multiplied or divided by a constant factor $b > 1$:

```

1 i = 1
2 while i < n:
3     # O(1) body operations
4     i *= 2

```

At iteration k , the variable has value $i = 2^k$. The loop terminates when $2^k \geq n \implies k = \lceil \log_2 n \rceil$. Thus, $T(n) = \Theta(\log n)$. Note that by the logarithm change-of-base rule ($\log_b n = \frac{\ln n}{\ln b}$), the base of the logarithm only contributes a constant factor, so we write $O(\log n)$ without specifying the base.

3.3 Triangular Nested Loops: $\Theta(n^2)$

When the inner loop boundary depends directly on the outer loop index:

```

1 for i in range(n):
2     for j in range(i, n):
3         # O(1) constant operation
4         process(i, j)

```

The inner loop executes $n - i$ times. The total operations sum as:

$$S(n) = \sum_{i=0}^{n-1} (n - i) = n + (n - 1) + (n - 2) + \dots + 1 = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

Neglecting lower-order terms and the scalar coefficient $\frac{1}{2}$, the running time is $\Theta(n^2)$.

3.4 Geometric Series Loop Progressions: $\Theta(n)$

Consider the following nested structure where the outer variable halves and the inner loop runs linearly up to the outer index:

```

1 def geometric_loop(n):
2     i = n
3     while i > 0:
4         for j in range(i):
5             print(j)
6         i //= 2

```

Mathematical Analysis: The outer loop takes values $i = n, \lfloor n/2 \rfloor, \lfloor n/4 \rfloor, \dots, 1$. The total operation count is bounded by an infinite geometric series:

$$S(n) = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots \leq n \sum_{k=0}^{\infty} \left(\frac{1}{2}\right)^k = n \cdot \frac{1}{1 - 1/2} = 2n$$

Since $S(n) \leq 2n$, the time complexity is strictly $\Theta(n)$, **not** $O(n \log n)$.

3.5 Harmonic Number Loops: $\Theta(n \log n)$

Consider a loop pattern frequently encountered in prime sieving (Sieve of Eratosthenes):

```

1 for i in range(1, n + 1):
2     j = i
3     while j <= n:
4         # O(1) step
5         j += i

```

For each i , the inner loop executes $\lfloor n/i \rfloor$ times. The total count is:

$$T(n) = \sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor \approx n \sum_{i=1}^n \frac{1}{i} = n \cdot H_n$$

where $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \ln n + \gamma + O(1/n)$ is the n -th Harmonic Number. Therefore, $T(n) = \Theta(n \log n)$.

3.6 Quadratic Loop Conditions: $\Theta(\sqrt{n})$

```

1 i = 1
2 while i * i <= n:
3     # O(1) operations
4     i += 1

```

The loop condition $i^2 \leq n$ is mathematically equivalent to $i \leq \sqrt{n}$. The loop executes $\lfloor \sqrt{n} \rfloor$ times, yielding time complexity $\Theta(\sqrt{n})$.

3.7 Best-Case, Worst-Case, and Average-Case Behavior

Algorithmic complexity often depends on the arrangement of input data:

- **Best Case** ($T_{\text{best}}(n)$): The minimum number of operations required across any input instance of size n . For Linear Search, finding the target at index 0 takes $\Omega(1)$ time.
- **Worst Case** ($T_{\text{worst}}(n)$): The maximum operations across any input instance of size n . For Linear Search, scanning the entire array when the item is absent takes $O(n)$ time.
- **Average Case** ($T_{\text{avg}}(n)$): The expected number of operations assuming a uniform probability distribution over all possible input permutations. For Linear Search:

$$T_{\text{avg}}(n) = \sum_{i=1}^n \frac{1}{n} \cdot i = \frac{1}{n} \frac{n(n+1)}{2} = \frac{n+1}{2} = \Theta(n)$$

4. Divide-and-Conquer Recurrences and the Master Theorem

4.1 Formulating Recurrence Relations

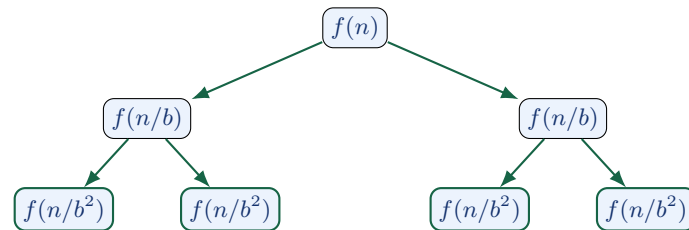
Divide-and-conquer algorithms partition a problem of size n into a smaller subproblems, each of size n/b , solve them recursively, and merge the partial solutions with combining cost $f(n)$. The general recurrence relation is:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- $a \geq 1$: Number of subproblems solved recursively at each level.
- $b > 1$: Factor by which the input size is reduced in each subproblem.
- $f(n)$: Cost of dividing the input and combining results outside the recursive calls.

4.2 The Recursion Tree Method

The recursion tree provides complete structural insight into how work accumulates across recursive levels:



1. **Tree Depth:** Subproblem size at level j is n/b^j . The base case is reached when $n/b^j = 1 \implies \text{Depth } D = \log_b n$.
2. **Number of Subproblems at Level j :** Exactly a^j nodes.
3. **Total Leaves at Depth $\log_b n$:** $a^{\log_b n} = n^{\log_b a}$ leaves.
4. **Leaf-Level Cost:** If base cases cost $\Theta(1)$, total leaf cost is $\Theta(n^{\log_b a})$.

4.3 The Standard Master Theorem (CLRS)

Theorem 4.1 (Master Theorem for Divide-and-Conquer)

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be an asymptotically positive function, and let $T(n)$ be defined on the integers by $T(n) = aT(n/b) + f(n)$. The asymptotic behavior of $T(n)$ is determined by comparing $f(n)$ to the watershed function $n^{\log_b a}$:

- **Case 1 (Leaf Dominant – Recursion Dominates):** If there exists a constant $\epsilon > 0$ such that:

$$f(n) = O\left(n^{\log_b a - \epsilon}\right)$$

then $f(n)$ is polynomially smaller than $n^{\log_b a}$, and:

$$T(n) = \Theta\left(n^{\log_b a}\right)$$

- **Case 2 (Balanced Work Distribution):** If $f(n) = \Theta\left(n^{\log_b a}\right)$, the cost is evenly distributed across all $\log_b n$ levels:

$$T(n) = \Theta\left(n^{\log_b a} \log n\right)$$

- **Case 3 (Root Dominant – Combine Cost Dominates):** If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$, and if $f(n)$ satisfies the **regularity condition**:

$$a \cdot f(n/b) \leq c \cdot f(n) \quad \text{for some constant } c < 1 \text{ and all sufficiently large } n,$$

then:

$$T(n) = \Theta(f(n))$$

The Requirement of a Polynomial Gap

In Cases 1 and 3, the difference between $f(n)$ and $n^{\log_b a}$ must be **polynomial** (a factor of n^ϵ for $\epsilon > 0$), not merely logarithmic. For example, in $T(n) = 2T(n/2) + n/\log n$, we have $n^{\log_2 2} = n$. The function $f(n) = n/\log n$ is asymptotically smaller than n , but the gap is $1/\log n$, which is sub-polynomial ($n^\epsilon > \log n$ for any $\epsilon > 0$). Hence, the standard 3-case Master Theorem cannot solve this recurrence.

4.4 The Extended Master Theorem (Polylogarithmic Form)

To resolve recurrences with logarithmic factors in $f(n)$, computer scientists utilize the extended formulation:

Theorem 4.2 (Extended Master Theorem)

For recurrences of the form $T(n) = aT(n/b) + \Theta(n^k \log^p n)$ with $a \geq 1, b > 1, k \geq 0$, compare a with b^k :

Condition	Log Exponent p	Solution $T(n)$	
$a > b^k$	Any $p \in \mathbb{R}$	$\Theta\left(n^{\log_b a}\right)$	(Recursion dominates)
$a = b^k$	$p > -1$	$\Theta\left(n^k \log^{p+1} n\right)$	(Standard balanced)
	$p = -1$	$\Theta\left(n^k \log \log n\right)$	(Boundary logarithmic case)
	$p < -1$	$\Theta\left(n^k\right)$	(Sum converges)
$a < b^k$	$p \geq 0$	$\Theta\left(n^k \log^p n\right)$	(Root cost dominates)
	$p < 0$	$O\left(n^k\right)$	

4.5 When the Master Theorem Cannot Be Applied (7 Classic Traps)

The Master Theorem is powerful, but fails when:

1. **Variable Subproblem Count:** a is not a constant (e.g., $T(n) = 2^n T(n/2) + n^n$).
2. **Non-Uniform Subproblem Sizes:** Subproblems have varying fractions (e.g., $T(n) = T(n/3) + T(2n/3) + n$). Requires Akra-Bazzi method or recursion tree.
3. **Sub-Polynomial Gap:** In $T(n) = 2T(n/2) + n/\log n$, Case 1 fails because $n/\log n$ is not $O(n^{1-\epsilon})$.
4. **Subproblem Fraction $b \leq 1$:** Subproblem sizes decrease by subtraction, not division (e.g., $T(n) = T(n-1) + n$).
5. **Violation of Regularity:** In Case 3, $f(n)$ oscillates or fails $af(n/b) \leq cf(n)$ (e.g., $T(n) = 2T(n/2) + n^2(2 + \cos n)$).
6. **Subproblem Count $a < 1$:** For example, $T(n) = 0.5T(n/2) + 1/n$.
7. **Non-Positive Combining Function:** $f(n)$ takes negative values (e.g., $T(n) = 64T(n/8) - n^2 \log n$).

4.6 Subtract-and-Conquer Recurrences

When problem size decreases by a fixed subtraction amount $b > 0$:

$$T(n) = aT(n-b) + f(n), \quad \text{where } f(n) = O(n^k)$$

Condition on a	Complexity $T(n)$	Representative Example
$a < 1$	$O(f(n)) = O(n^k)$	$T(n) = 0.5T(n-1) + n^2 \implies O(n^2)$
$a = 1$	$O(n \cdot f(n)) = O(n^{k+1})$	$T(n) = T(n-1) + n \implies O(n^2)$ (Selection Sort)
$a > 1$	$O(f(n) \cdot a^{n/b}) = O(n^k a^{n/b})$	$T(n) = 2T(n-1) + 1 \implies O(2^n)$ (Towers of Hanoi)

4.7 Advanced Recurrences via Variable Substitution

Solving $T(n) = \sqrt{n} \cdot T(\sqrt{n}) + n$

This recurrence cannot be solved by the Master Theorem because $a = \sqrt{n}$ is not constant.

1. Divide both sides by n :

$$\frac{T(n)}{n} = \frac{\sqrt{n} \cdot T(\sqrt{n})}{n} + 1 = \frac{T(\sqrt{n})}{\sqrt{n}} + 1$$

2. Define substitution variable $U(n) = \frac{T(n)}{n}$. The relation becomes:

$$U(n) = U(\sqrt{n}) + 1$$

3. Track input shrinkage: $n \rightarrow n^{1/2} \rightarrow n^{1/4} \rightarrow \dots \rightarrow n^{1/2^k}$.
4. The process terminates when subproblem size reaches a constant $c = 2$:

$$n^{1/2^k} = 2 \implies \frac{1}{2^k} \log_2 n = 1 \implies 2^k = \log_2 n \implies k = \log_2(\log_2 n)$$

5. Each step adds 1, so $U(n) = \Theta(\log \log n)$.
6. Re-substituting $T(n) = n \cdot U(n)$ yields:

$$T(n) = \Theta(n \log \log n)$$

5. Amortized Analysis Foundations & Case Studies

5.1 The Motivation for Amortized Analysis

Traditional worst-case analysis evaluates the maximum cost of an individual operation in isolation. However, in many sophisticated data structures:

- An expensive operation cannot occur in isolation; it requires a long sequence of prior cheap operations to set up.
- Evaluating every operation by the worst-case single step produces an overly pessimistic bound.

Definition 5.1 (Amortized Cost)

In a sequence of m operations, if the worst-case total running time across the entire sequence is $T(m)$, the **amortized cost** per operation is:

$$\hat{c} = \frac{T(m)}{m}$$

Key Distinction: Amortized analysis guarantees an upper bound on the average performance of each operation in the *worst-case sequence*. It involves **no probabilistic assumptions** about the input distribution (unlike average-case analysis).

5.2 The Three Standard Amortized Analysis Techniques

5.2.1 1. The Aggregate Method

Show that for all m , a sequence of m operations takes total time $T(m)$ in the worst case. The amortized cost per operation is directly computed as $\hat{c} = T(m)/m$.

5.2.2 2. The Accounting Method (Banker's Method)

Assign an artificial charge (**amortized cost** $\hat{c}_i$) to each operation type:

- Cheap operations are overcharged ($\hat{c}_i > c_i$), and the surplus is stored as “credit” on specific data structure elements.
- Expensive operations are undercharged ($\hat{c}_i < c_i$), and draw from the accumulated credit to pay for their actual cost.
- **Validity Invariant:** Total stored credit must remain non-negative at all times:

$$\sum_{i=1}^m \hat{c}_i \geq \sum_{i=1}^m c_i \quad \text{for all } m \geq 1.$$

5.2.3 3. The Potential Method (Physicist's Method)

Define a **potential function** $\Phi(D)$ mapping the data structure state D to a real number $\mathbb{R}$:

- $\Phi(D_0) = 0$ (initial state potential is 0).
- $\Phi(D_i) \geq 0$ for all valid states D_i (potential is always non-negative).
- The amortized cost of the i -th operation is defined as:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = c_i + \Delta\Phi_i$$

- Summing across all m operations telescopes the potential terms:

$$\sum_{i=1}^m \hat{c}_i = \sum_{i=1}^m (c_i + \Phi(D_i) - \Phi(D_{i-1})) = \sum_{i=1}^m c_i + \Phi(D_m) - \Phi(D_0) \geq \sum_{i=1}^m c_i$$

5.3 Case Study 1: Dynamic Array (Python ‘list’) Resizing

Consider a dynamic array that doubles its capacity whenever an element is appended to a full array:

```

1 class DynamicArray:
2     def __init__(self):
3         self.capacity = 1
4         self.size = 0
5         self.data = [None] * self.capacity
6
7     def append(self, x):
8         if self.size == self.capacity:
9             self._resize(2 * self.capacity)
10        self.data[self.size] = x
11        self.size += 1
12
13    def _resize(self, new_capacity):
14        new_data = [None] * new_capacity
15        for i in range(self.size):
16            new_data[i] = self.data[i]
17        self.data = new_data
18        self.capacity = new_capacity
    
```

5.3.1 Analysis via Aggregate Method

- An insertion that requires no resize costs 1 operation.
- Array expansions occur when $n = 1, 2, 4, 8, 16, \dots, 2^k$.
- The total number of element copying operations across n insertions is:

$$\sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j = 2^{\lfloor \log_2 n \rfloor + 1} - 1 < 2n$$

- Total cost for n operations is $T(n) = n$ (insertions) + $2n$ (copies) $\leq 3n$.
- Amortized Cost:** $\hat{c} = \frac{T(n)}{n} \leq \frac{3n}{n} = 3 = \Theta(1)$.

Doubling versus Fixed Increments

If the array increased its capacity by a fixed constant $+k$ (e.g., +1000 elements) instead of doubling, an array reaching size n would undergo $\approx n/k$ resizes, copying $\sum_{i=1}^{n/k} i \cdot k \approx \Theta(n^2)$ elements. The amortized cost per append would degrade to $\Theta(n)$! Geometric doubling is mandatory to achieve $O(1)$ amortized insertion.

5.3.2 Analysis via Potential Method

Let s_i be the number of items and c_i be the capacity after the i -th operation. Define the potential function:

$$\Phi(D_i) = 2s_i - c_i$$

- Non-negativity:** Immediately after a doubling, $c_i = 2s_i \implies \Phi(D_i) = 0$. When full, $c_i = s_i \implies \Phi(D_i) = s_i \geq 0$.

- **Case A (No Resize):** $c_i = c_{i-1}$, $s_i = s_{i-1} + 1$.

$$\hat{c}_i = c_i + \Delta\Phi_i = 1 + (2(s_{i-1} + 1) - c_{i-1}) - (2s_{i-1} - c_{i-1}) = 1 + 2 = 3$$

- **Case B (With Resize):** Array was full ($s_{i-1} = c_{i-1}$). New capacity $c_i = 2c_{i-1}$. Actual cost $c_i = s_{i-1} + 1$.

$$\Delta\Phi_i = (2(s_{i-1} + 1) - 2s_{i-1}) - (2s_{i-1} - s_{i-1}) = 2 - s_{i-1}$$

$$\hat{c}_i = (s_{i-1} + 1) + (2 - s_{i-1}) = 3$$

In both cases, $\hat{c}_i = 3 = O(1)$. The amortized cost per append is strictly $O(1)$.

5.4 Case Study 2: Stack with MultiPop

Consider a stack data structure with standard ‘Push(x)’ ($O(1)$), ‘Pop()’ ($O(1)$), and ‘MultiPop(k)’ which pops $\min(k, \text{size})$ items:

```

1 def multipop(stack, k):
2     while stack and k > 0:
3         stack.pop()
4         k -= 1

```

Analysis: A single ‘MultiPop’ operation can cost $O(n)$ if the stack contains n elements. However, an element can only be popped if it was previously pushed. In any sequence of m operations starting with an empty stack, the total number of pops cannot exceed the total number of pushes (which is $\leq m$). Hence, total time is at most $2m$, yielding an **amortized cost of $O(1)$ per operation**.

6. Worked Problems and Solutions

6.1 Problem 1: Growth Rate Ranking

Problem: Rank the following functions in order of asymptotic growth rate from slowest to fastest. Group functions that belong to the same Θ -class:

$$f_1(n) = n^{1.5}, \quad f_2(n) = 1000n, \quad f_3(n) = n \log_2 n, \quad f_4(n) = 2^{\sqrt{n}}, \quad f_5(n) = n!, \quad f_6(n) = 2^n, \quad f_7(n) = n^2$$

Solution to Problem 1

Taking logarithms or evaluating limits:

1. $f_2(n) = 1000n = \Theta(n)$
2. $f_3(n) = n \log_2 n$ (grows faster than n , slower than $n^{1.5}$)
3. $f_1(n) = n^{1.5} = n^{3/2}$
4. $f_7(n) = n^2$
5. $f_4(n) = 2^{\sqrt{n}}$: compare with n^2 . $\log_2(2^{\sqrt{n}}) = \sqrt{n}$, whereas $\log_2(n^2) = 2 \log_2 n$. Since $\sqrt{n} \succ \log n$, $2^{\sqrt{n}}$ eventually dominates all polynomials! Thus $n^2 \prec 2^{\sqrt{n}} \prec 2^n$.
6. $f_6(n) = 2^n$
7. $f_5(n) = n! \approx \sqrt{2\pi n}(n/e)^n$ (Stirling's approximation, dominates 2^n).

Final Ranking:

$$1000n \prec n \log n \prec n^{1.5} \prec n^2 \prec 2^{\sqrt{n}} \prec 2^n \prec n!$$

6.2 Problem 2: Non-Trivial Loop Complexity

Problem: Compute the tight asymptotic bound Θ for the following Python function:

```

1 def compute(n):
2     total = 0
3     i = 1
4     while i <= n:
5         j = 1
6         while j <= n:
7             total += 1
8             j *= 2
9         i += 1
10    return total

```

Solution to Problem 2

- The outer loop initializes $i = 1$ and increments $i \leftarrow i + 1$ until $i > n$. Thus, the outer loop executes exactly n times.
- For every single outer iteration, the inner loop initializes $j = 1$ and doubles $j \leftarrow j \times 2$ until $j > n$. The sequence of j values is $1, 2, 4, 8, \dots, 2^k$. The inner loop terminates when $2^k > n \implies k = \lfloor \log_2 n \rfloor + 1$. The inner loop executes $\Theta(\log n)$ times independently of i .
- By the Product Rule: $T(n) = \sum_{i=1}^n \Theta(\log n) = n \cdot \Theta(\log n) = \Theta(n \log n)$.

6.3 Problem 3: Applying the Master Theorem

Problem: Solve the following divide-and-conquer recurrence relations:

1. $T_a(n) = 8T_a(n/2) + n^2$
2. $T_b(n) = 2T_b(n/2) + n \log^2 n$
3. $T_c(n) = 3T_c(n/4) + n \log n$

Solution to Problem 3

Part 1: For $T_a(n) = 8T_a(n/2) + n^2$:

- $a = 8, b = 2, f(n) = n^2$.
- Benchmark: $n^{\log_b a} = n^{\log_2 8} = n^3$.
- Compare $f(n) = n^2$ with n^3 : Since $n^2 = O(n^{3-1})$ with $\epsilon = 1 > 0$, this falls into **Case 1**.
- **Conclusion:** $T_a(n) = \Theta(n^3)$.

Part 2: For $T_b(n) = 2T_b(n/2) + n \log^2 n$:

- $a = 2, b = 2, f(n) = n \log^2 n$.
- Benchmark: $n^{\log_2 2} = n^1$. Here $k = 1$ and $p = 2$.
- Since $a = b^k$ ($2 = 2^1$) and $p = 2 > -1$, this falls into **Extended Case 2**.
- **Conclusion:** $T_b(n) = \Theta(n^k \log^{p+1} n) = \Theta(n \log^3 n)$.

Part 3: For $T_c(n) = 3T_c(n/4) + n \log n$:

- $a = 3, b = 4, f(n) = n \log n$.
- Benchmark: $n^{\log_4 3} \approx n^{0.7925}$.
- Here $f(n) = n \log n = \Omega(n^{0.7925+\epsilon})$ for $\epsilon \approx 0.2$.
- Regularity check: $af(n/b) = 3(n/4 \log(n/4)) \leq \frac{3}{4}n \log n$. Choosing $c = 0.8 < 1$, regularity holds.
- This is **Case 3**.
- **Conclusion:** $T_c(n) = \Theta(n \log n)$.

7. Quick Reference Summary Sheet

7.1 Essential Summation Formulas

- **Arithmetic Series:** $\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2} = \Theta(n^2)$
- **Sum of Squares:** $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$
- **Finite Geometric Series** ($r \neq 1$): $\sum_{i=0}^k r^i = \frac{r^{k+1}-1}{r-1}$
- **Infinite Geometric Series** ($|r| < 1$): $\sum_{i=0}^{\infty} r^i = \frac{1}{1-r}$
- **Harmonic Series:** $\sum_{i=1}^n \frac{1}{i} = \ln n + \gamma + O(1/n) = \Theta(\log n)$
- **Stirling's Factorial Approximation:** $n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + O(1/n)) \implies \log(n!) = \Theta(n \log n)$

7.2 Master Theorem Decision Matrix

For recurrences $T(n) = aT(n/b) + \Theta(n^k \log^p n)$:

Comparison	Exponent p	Dominant Factor	Solution $T(n)$
$a > b^k$	Any p	Leaves (Depth $\log_b n$)	$\Theta(n^{\log_b a})$
$a = b^k$	$p > -1$	All Levels Equal	$\Theta(n^k \log^{p+1} n)$
$a = b^k$	$p = -1$	Boundary	$\Theta(n^k \log \log n)$
$a = b^k$	$p < -1$	Leaf/Root	$\Theta(n^k)$
$a < b^k$	$p \geq 0$	Root (Combine Cost)	$\Theta(n^k \log^p n)$
$a < b^k$	$p < 0$	Root	$O(n^k)$

7.3 Amortized Cost Comparison Matrix

Operation	Worst-Case	Amortized	Key Mechanism
Dynamic Array 'append'	$O(n)$ (during resize)	$O(1)$	Geometric doubling ensures total re-size copies $< 2n$
Stack 'MultiPop(k)'	$O(k)$	$O(1)$	Elements cannot be popped more times than pushed
k -bit Counter Increment	$O(k)$	$O(1)$	Bit i flips once every 2^i operations: $\sum 1/2^i \leq 2$
Hash Table Insert	$O(n)$ (rehash)	$O(1)$	Capacity doubling with uniform hash distribution

End of Lecture Notes – Part 03: Algorithms and Complexity Analysis

Department of Data Science & AI · National Economics University